

Adjustments to Union Lifetime Rules

Document #: P3726R2 [[Latest](#)] [[Status](#)]
Date: 2026-03-26
Project: Programming Language C++
Audience: CWG, LWG
Reply-to: Barry Revzin
<barry.revzin@gmail.com>
Tomasz Kamiński
<tomaszkam@gmail.com>

Contents

- [1 Revision History](#)
- [2 Introduction](#)
- [3 Proposal](#)
 - [3.1 Fixing When Implicit Lifetime Starts](#)
 - [3.2 Fixing Which Values are Constituent Values](#)
- [4 Wording](#)
 - [4.1 Feature-Test Macro](#)
- [5 Acknowledgments](#)
- [6 References](#)

§ 1 Revision History

Since [\[P3726R1\]](#), added feature-test macro and some wording improvements.

[\[P3726R0\]](#) proposed several things:

1. Reverting the P3074 rule implicitly starting lifetime of the first alternative in a union
2. Having placement-new on an aggregate element implicitly start the lifetime of the aggregate
3. extending the constituent value rule to allow array elements of a union that are not within their lifetime.

Following [Core review in Kona](#), this revision keeps (1), changes (2) to instead use the solution proposed in [\[P3074R0\] \(constexpr union lifetime\)](#), and while Core wanted to significantly extend the rule in (3), we choose to keep it as-is and instead extend our reasoning for this being the correct rule.

§ 2 Introduction

[\[P3074R7\] \(trivial unions \(was std::uninitialized<T>\)\)](#) was adopted in Hagenberg. One of the goals of that paper was to make an example like this work:

```

template <typename T, size_t N>
struct FixedVector {
    union { T storage[N]; };
    size_t size = 0;

    constexpr FixedVector() = default;

    constexpr ~FixedVector() {
        std::destroy(storage, storage + size);
    }

    constexpr auto push_back(T const& v) -> void {
        ::new (storage + size) T(v);
        ++size;
    }
};

constexpr auto silly_test() -> size_t {
    FixedVector<std::string, 3> v;
    v.push_back("some sufficiently longer string");
    return v.size;
}

static_assert(silly_test() == 1);

```

That paper solved this problem by:

1. Making `unions` have trivial default constructors and trivial destructors, by default, and
2. Implicitly starting the lifetime of the first `union` member, if that member has implicit-lifetime type.

The first avoids pointless no-op empty constructors and destructors, and the second would start the lifetime of the `T[N]` member — which we need in order for the placement new to be well-defined. Which seemed to be a pretty nice thing, as the code just works, without any complicated intervention.

However, there is an important principle in C++ language design that Barry missed: we can't have nice things because there are no nice things. Richard Smith sent out this example:

```

union U { int a, b; };
template<U u> class X {};
constexpr U make() { U u; return u; }
void f(X<make(>)>) {}

```

He pointed out that today this is valid because template argument of `X` is a union object with no active member. But with the [\[P3074R7\]](#) changes, this:

1. Causes the union object to have an active member, and thus would have to be mangled differently. That change makes this an ABI break (although not all implementations mangle these cases differently, which is probably a bug, so the potential damage of ABI break isn't tremendous)

2. Causes the example to fail to compile because it's no longer a valid template argument.

The relevant rule here is 7.7 [\[expr.const\]](#) which says that:

- 22 A *constant expression* is either a glvalue core constant expression that refers to an object or a non-immediate function, or a prvalue core constant expression whose result object ([basic.lval]) satisfies the following constraints:
- (22.1) — each constituent reference refers to an object or a non-immediate function,
 - (22.2) — no constituent value of scalar type is an indeterminate or erroneous value ([basic.indet]),
 - (22.3) — [...]

where, in that same section:

- 2 The *constituent values* of an object *o* are
- (2.1) — if *o* has scalar type, the value of *o*;
 - (2.2) — otherwise, the constituent values of any direct subobjects of *o* other than inactive union members.

Note that inactive union members are excluded, but active-yet-uninitialized union members are disallowed.

This is also a problem because the original goal of the paper was to make types like `std::inplace_vector` completely usable at compile-time, and this rule (even separate from [\[P3074R7\]](#)) makes that impossible:

```
constexpr std::inplace_vector<int, 4> v = {1, 2};
```

A “normal” implementation would have `union { int storage[4]; }`, of which the first two elements are initialized but the last two are not. And thus have indeterminate value, so this isn't a valid constant expression. For `int` specifically (and trivial types more broadly), this is fixable by having the implementation simply have a `int storage[4];` instead and initialize all the objects — since that's free. But for types that either aren't trivially default constructible or aren't trivially destructible, that's not an option, and that really shouldn't be the limiting factor of whether you can create `constexpr` variables (or non-type template arguments) of such types.

We're hoping to fix both of those issues in this paper, with two fairly independent fixes.

§ 3 Proposal

This proposal comes in two parts.

§ 3.1 Fixing When Implicit Lifetime Starts

[P3074R7] added this wording to 11.4.5.2 [class.default.ctor]:

4 If a default constructor of a union-like class X is trivial, then for each union U that is either X or an anonymous union member of X, if the first variant member, if any, of U has implicit-lifetime type ([basic.types.general]), the default constructor of X begins the lifetime of that member if it is not the active member of its union. [*Note 1*: It is already the active member if U was value-initialized. — *end note*] ~~An~~ Otherwise, an implicitly-defined ([dcl.fct.def.default]) default constructor performs the set of initializations of the class that would be performed by a user-written default constructor for that class with no ctor-initializer ([class.base.init]) and an empty compound-statement.

That wording needs to be reverted. The default constructor will no longer start lifetimes implicitly.

The previous version of this paper [P3726R0] proposed allowing placement new on an aggregate element to start the lifetime of the aggregate. It turns out that the wording to allow this was wholly incomplete, as it didn't account for:

- casts to `void*`
- calls to `std::addressof`

As a production implementation wouldn't write `new (&storage[n]) T`, it'd write `::new ((void*)std::addressof(storage[n])) T`. Having to pattern match on syntax makes this wording approach increasingly complicated, if not outright weird given the call to a standard library function in there. It also makes for a confusing design since you would get into situations where `::new (E) T`; is valid but `auto ptr = E; ::new (ptr) T`; is not.

Instead, we go back to what [P3074R0]_(constexpr union lifetime) proposed: a dedicated standard library function start the lifetime of an implicit-lifetime time. That is, our `FixedVector` implementation would look like this:

```
template <typename T, size_t N>
struct FixedVector {
    union { T storage[N]; };
    size_t size = 0;

    constexpr FixedVector() {
        // explicitly starts the lifetime of the T[N]
        // does not invoke any constructors
        // no array element starts its lifetime yet
        // this becomes the active member of the union
        std::start_lifetime(storage);
    }

    constexpr auto push_back(T const& v) -> void {
```

```

        // Now, this is always okay because storage is within its
        lifetime
        ::new (storage + size) T(v);
        ++size;
    }
};

```

Note that the previous revision of this paper had to consider the problem of template-argument equivalences because a default-constructed `FixedVector` would not yet have started `storage`'s lifetime, so two empty vectors could be in different states. But that is no longer an issue here since `storage`'s lifetime is always started on construction. This makes it so that all empty `FixedVectors` are equivalent: they both have an in-lifetime storage member with 0 active elements.

We propose the signature:

```

template<class T>
constexpr void start_lifetime(T& r) noexcept;

```

Where `T` is restricted to be an implicit-lifetime aggregate. We need `T` to be implicit-lifetime for all lifetime starting reasons, but we also want a narrower restriction that `T` is specifically an aggregate type because we're really not invoking any constructors here. Also, this is notably distinct from `std::start_lifetime_as<T>` since that function also recursively begins the lifetime of all implicit-lifetime subobjects, and in this context we need that to not happen. This function will only start the lifetime of the top-level object. Perhaps this function could eventually be extended to also allow implicit-lifetime non-aggregates, but for now we think we should start narrow.

Additionally, this function takes a `T&` rather than `T*`. The other `start_lifetime` functions take a `void*` — because that is how they are expected to be used. `std::start_lifetime_as<T>` brings forth a `T*` out of a buffer. But this function needs to be run during constant evaluation, where you already have an object at hand, just not one within its lifetime. So taking a reference is more suitable for this problem.

§ 3.2 Fixing Which Values are Constituent Values

The current rule for constituent values is, from 7.7 [\[expr.const\]/2](#):

2 The *constituent values* of an object *o* are

- (2.1) — if *o* has scalar type, the value of *o*;
- (2.2) — otherwise, the constituent values of any direct subobjects of *o* other than inactive union members.

As mentioned earlier, this means that if we have a `union { T storage[4]; }` then either there are no constituent values (if `storage` is inactive) or we consider all of the `T`s as constituent values (even if we only constructed the first two). So we'll need to loosen this rule to permit objects with union members to be more usable as constant expressions.

For the FixedVector (aka static_vector aka inplace_vector) example, we really only need to allow “holes” at the end of the array. But if we want to support a different container, that is more bidirectional and supports cheap push_front and pop_front, we will also want to support “holes” at the front of the array. So for simplicity, we’re proposing to support holes *anywhere* in the array. Note that we’re still not proposing nice syntax for actually constructing such an array with holes. Richard on the reflector had suggested a strawperson syntax:

```
// short array initializer:  
// initializes arr[0] and arr[1],  
// does not start lifetime of rest  
int arr[42] = {a, b, short};  
  
// in std::allocator<T>::allocate:  
return new (ptr) T[n]{short};
```

I don’t think we strictly need to solve that problem right now, but at least we can put in the groundwork for supporting it in the future.

Until then, we’re specifically proposing to extend the constituent values rule, which currently reads like this:

2 The *constituent values* of an object *o* are

- (2.1) — if *o* has scalar type, the value of *o*;
- (2.2) — otherwise, the constituent values of any direct subobjects of *o* other than inactive union members.

To instead read more like this:

2 The *constituent references* of an object *o* are

- (2.3) — any direct members of *o* that have reference type, and
- (2.4) — the constituent references of any direct subobjects of *o* other than inactive union members subobjects (see below).

An *inactive union subobject* is either

- (2.5) — an inactive union member or
- (2.6) — an element *E* of an array member of a union where *E* is not within its lifetime.

We *specifically* want to extend this *only* to array members of unions. This is the fix necessary to ensure that this:

```
constexpr std::inplace_vector<int, 4> v = {1, 2};
```

is a valid constexpr variable if the implementation uses a `union { int storage[4]; }` to hold the data, because we would only consider the first two elements of `storage` as constituent values — the fact that the last two elements are uninitialized no longer counts against us when we consider whether `v` is a valid result of a constant expression.

Core in Kona had expressed a view that this should be extended first to any aggregate (not just arrays) but also extended even further than that to allow for any aggregate *outside* of unions. We see no motivation for this expansion, and since it extends the cases where we have an incomplete constexpr variable, we'd like to see motivation for the expansion first. Additionally, what we are proposed here is consistent with the language we have today: we already have a notion of incomplete arrays. `std::allocator<T>::allocate` already has [similar behavior](#):

- 5 *Remarks:* The storage for the array is obtained by calling `::operator new` ([new.delete]), but it is unspecified when or how often this function is called. This function starts the lifetime of the array object, but not that of any of the array elements.

Allowing incomplete arrays within a union is pretty analogous to allowing incomplete arrays that are heap-allocated, and is indeed how `std::vector` works during constant evaluation. When we do eventually get non-transient constexpr allocation, it will also be how constexpr `std::vectors` work. For both types, copying and destruction is handled manually — there are no problems caused by a normal destructor attempting to destroy partial objects.

Put differently, supporting the array case is necessary, useful, consistent, and already needs to be supported — but supporting wider cases doesn't meet this bar for us. Especially this late into the process for C++26. This restriction can always be relaxed in C++29, if sufficient motivation is presented.

§ 4 Wording

Change to 7.7 [\[expr.const\]](#):

* A *union elemental subobject* is a direct member of a union or an element of an array that is a union elemental subobject. An *inactive union elemental subobject* is a union elemental subobject that is not within its lifetime.

2 The *constituent values* of an object *o* are

(2.1) — if *o* has scalar type, the value of *o*;

(2.2) — otherwise, the constituent values of any direct subobjects of *o* other than inactive union ~~members~~ elemental subobjects.

The *constituent references* of an object *o* are

- (2.3) — any direct members of *o* that have reference type, and
- (2.4) — the constituent references of any direct subobjects of *o* other than inactive union members elemental subobjects.

[Example 1:

```
struct A {
    struct X {
        int i;
        int j;
    };

    struct Y {
        X x1;
        X x2;
    };

    union {
        int i;
        int arr[4];
        Y y;
    };
};

constexpr A v1;           // ok, no constituent values
constexpr A v2{.i=1};    // ok, the constituent values are {v2.i}
constexpr A v3 = []{
    A a;
    std::start_lifetime(a.arr); // ok, arr is now the active element of
                                // the union
    new (&a.arr[1]) int(1);
    a.arr[2] = 2;
    return a;
}();                       // ok, the constituent values are {v3.arr[1],
                            // v3.arr[2]}
constexpr A v4 = []{
    A a;
    a.y.x1.i = 1;
    a.y.x2.j = 2;
    return a;
}();                       // error: the constituent values include v4.y.x1.j
                            // and v4.y.x2.i
                            // which have erroneous value
```

— end example]

Revert the change in 11.4.5.2 [class.default.ctor]/4:

- 4 If a default constructor of a union-like class *X* is trivial, then for each union *U* that is either *X* or an anonymous union member of *X*, if the first variant member, if any, of *U* has implicit-lifetime type ([basic.types.general]), the default constructor of *X* begins the lifetime of that member if it is not the active member of its union. [Note 1: It is already the active member if *U* was value-initialized. — end note] An Otherwise, an implicitly-defined

([dcl.fct.def.default]) default constructor performs the set of initializations of the class that would be performed by a user-written default constructor for that class with no ctor-initializer ([class.base.init]) and an empty compound-statement.

Extend the template-argument-equivalent rules to understand not-within-lifetime arrays, in 13.6 [\[temp.-type\]](#):

- 2 Two values are *template-argument-equivalent* if they are of the same type and
- (2.1) — [...]
- (2.8) — they are of array type and their corresponding elements are either both within their lifetimes and template-argument-equivalent or both not within their lifetimes, or
- (2.9) — [...]

Add to 20.2.2 [\[memory.syn\]](#):

```
namespace std {
    // ...
    // [obj.lifetime], explicit lifetime management
+ template<class T>
+         constexpr void start_lifetime(T& r) noexcept;
    // freestanding
    template<class T>
        T* start_lifetime_as(void* p) noexcept;
    // freestanding
    template<class T>
        const T* start_lifetime_as(const void* p) noexcept;
    // freestanding
    template<class T>
        volatile T* start_lifetime_as(volatile void* p) noexcept;
    // freestanding
    template<class T>
        const volatile T* start_lifetime_as(const volatile void* p) noexcept;
    // freestanding
    template<class T>
        T* start_lifetime_as_array(void* p, size_t n) noexcept;
    // freestanding
    template<class T>
        const T* start_lifetime_as_array(const void* p, size_t n) noexcept;
    // freestanding
    template<class T>
        volatile T* start_lifetime_as_array(volatile void* p, size_t n)
        noexcept; // freestanding
    template<class T>
        const volatile T* start_lifetime_as_array(const volatile void* p,
        // freestanding
        size_t n) noexcept;
}
```

With corresponding wording in 20.2.6 [obj.lifetime] [Drafting note: We specifically want to allow implicit-lifetime *aggregate* types only. Implicit-lifetime types are still allowed to have constructors, and the combination of implicit-lifetime and aggregate means that there is neither a user-provided constructor nor a user-provided destructor — which means that starting lifetime in this manner is reasonable, we are not side-stepping any functionality.]:

```
template<class T>
    constexpr void start_lifetime(T& r) noexcept;
```

1 *Mandates:* *T* is a complete type and an implicit-lifetime ([basic.type]) aggregate ([dcl.init.aggr]) type.

2 *Effects:* If the object referenced by *r* is already within its lifetime ([basic.life]), no effects. Otherwise, begins the lifetime of the object referenced by *r*. [*Note 2:* No initialization is performed and no subobject has its lifetime started. If *r* denotes a member of a union *U*, it becomes the active member of *U* ([class.union]). — *end note*]

§ 4.1 Feature-Test Macro

Bump the feature-test macro added by [P3074R7]:

```
- __cpp_trivial_union 202502L
+ __cpp_trivial_union 2026XXL
```

And add a new library macro in 17.3.2 [version.syn]

```
+ #define __cpp_lib_start_lifetime 2026XXL // also in <memory>
```

§ 5 Acknowledgments

Thank you to Richard Smith for bringing the issue to our attention and for all the helpful suggestions. Thank you to Tim Song for all the help on this topic.

§ 6 References

[P3074R0] Barry Revzin. 2023-12-15. constexpr union lifetime.

<https://wg21.link/p3074r0>

[P3074R7] Barry Revzin. 2025-02-14. trivial unions (was std::uninitialized<T>).

<https://wg21.link/p3074r7>

[P3726R0] Barry Revzin, Tomasz Kamiński. 2025-07-13. Adjustments to Union Lifetime Rules.

<https://wg21.link/p3726r0>

[P3726R1] Barry Revzin, Tomasz Kamiński. 2025-12-15. Adjustments to Union Lifetime Rules.
<https://wg21.link/p3726r1>